

Transaction Processing

The concept of transaction provides a mechanism for describing logical units of database processing. Transaction processing systems are systems with large databases and hundreds of concurrent users executing database transactions. Examples:

- airline reservations
- banking
- credit card processing,
- online retail purchasing,
- Stock markets, supermarket checkouts, and many other applications

These systems require high availability and fast response time for hundreds of concurrent users. A transaction is typically implemented by a computer program, which includes database commands such as retrievals, insertions, deletions, and updates.

5.1 Objectives

- ❖ To study transaction properties
- ❖ To study creation of schedule and maintaining schedule equivalence.
- ❖ To check whether the given schedule is serializable or not.
- ❖ To study protocols used for locking objects
- ❖ Differentiating between 2PL and Strict 2PL

5.2 Introduction to Transaction Processing

5.2.1 Single-User versus Multiuser Systems

- One criterion for classifying a database system is according to the number of users who can use the system **concurrently**

Single-User versus Multiuser Systems

- A DBMS is
 - **single-user**
 - at most one user at a time can use the system
 - Eg: Personal Computer System
 - **multiuser**
 - many users can use the system and hence access the database concurrently
 - Eg: Airline reservation database

- Concurrent access is possible because of **Multiprogramming**. Multiprogramming can be achieved by:
 - interleaved execution
 - Parallel Processing
- Multiprogramming operating systems** execute some commands from one process, then suspend that process and execute some commands from the next process, and so on
- A process is resumed at the point where it was suspended whenever it gets its turn to use the CPU again
- Hence, concurrent execution of processes is actually **interleaved**, as illustrated in Figure 21.1

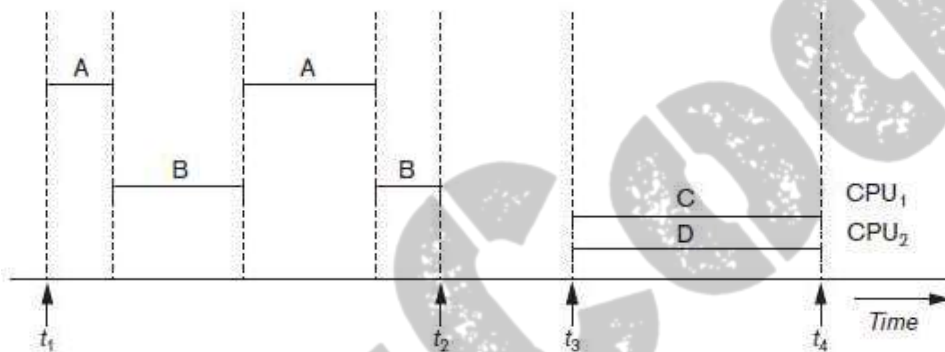


Figure 21.1
Interleaved processing versus parallel processing of concurrent transactions.

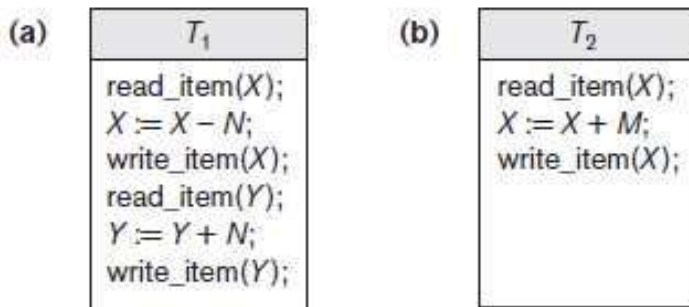
- Figure 21.1, shows two processes, A and B, executing concurrently in an interleaved fashion
- Interleaving keeps the CPU busy when a process requires an input or output (I/O) operation, such as reading a block from disk
- The CPU is switched to execute another process rather than remaining idle during I/O time
- Interleaving also prevents a long process from delaying other processes.
- If the computer system has multiple hardware processors (CPUs), **parallel processing** of multiple processes is possible, as illustrated by processes C and D in Figure 21.1
- Most of the theory concerning concurrency control in databases is developed in terms of **interleaved concurrency**
- In a multiuser DBMS, the stored data items are the primary resources that may be accessed concurrently by interactive users or application programs, which are constantly retrieving information from and modifying the database.

5.2.2 Transactions, Database Items, Read and Write Operations, and DBMS

Buffers

- A Transaction an executing program that forms a logical unit of database processing
- It includes one or more DB access operations such as insertion, deletion, modification or retrieval operation.
- It can be either embedded within an application program using **begin transaction** and **end transaction** statements Or specified interactively via a high level query language such as SQL
- Transaction which do not update database are known as **read only transactions**.
- Transaction which do update database are known as **read write transactions**.
- A **database** is basically represented as a collection of named data items The size of a data item is called its **granularity**.
- A **data item** can be a database record, but it can also be a larger unit such as a whole disk block, or even a smaller unit such as an individual field (attribute) value of some record in the database
- Each data item has a unique name
- **Basic DB access operations that a transaction can include are:**
 - **read_item(X)**: Reads a DB item named X into a program variable.
 - **write_item(X)**: Writes the value of a program variable into the DB item named X
- **Executing read_item(X) include the following steps:**
 1. Find the address of the disk block that contains item X
 2. Copy the block into a buffer in main memory
 3. Copy the item X from the buffer to program variable named X.
- **Executing write_item(X) include the following steps:**
 1. Find the address of the disk block that contains item X
 2. Copy the disk block into a buffer in main memory
 3. Copy item X from program variable named X into its correct location in buffer.
 4. Store the updated disk block from buffer back to disk (either immediately or later).
- Decision of when to store a modified disk block is handled by **recovery manager** of the DBMS in cooperation with operating system.
- A DB cache includes a number of data buffers.
- When the buffers are all occupied a buffer replacement policy is used to choose one of the buffers to be replaced. EG: LRU

- A transaction includes read_item and write_item operations to access and update DB.

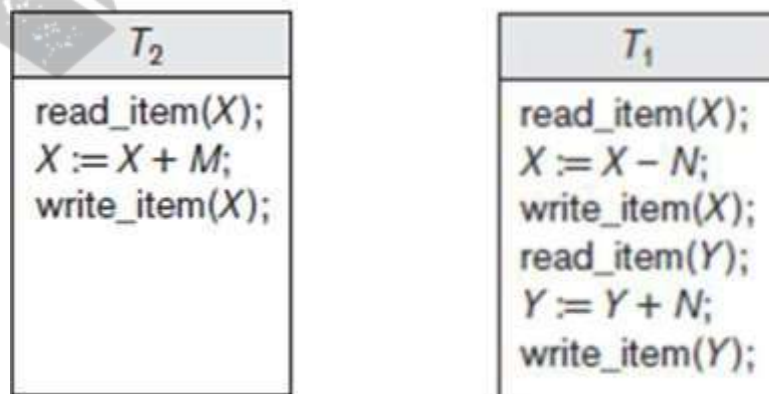
**Figure 21.2**

Two sample transactions. (a) Transaction T_1 . (b) Transaction T_2 .

- The **read-set** of a transaction is the set of all items that the transaction reads
- The **write-set** is the set of all items that the transaction writes
- For example, the read-set of T_1 in Figure 21.2 is $\{X, Y\}$ and its write-set is also $\{X, Y\}$.

5.2.3 Why Concurrency Control Is Needed

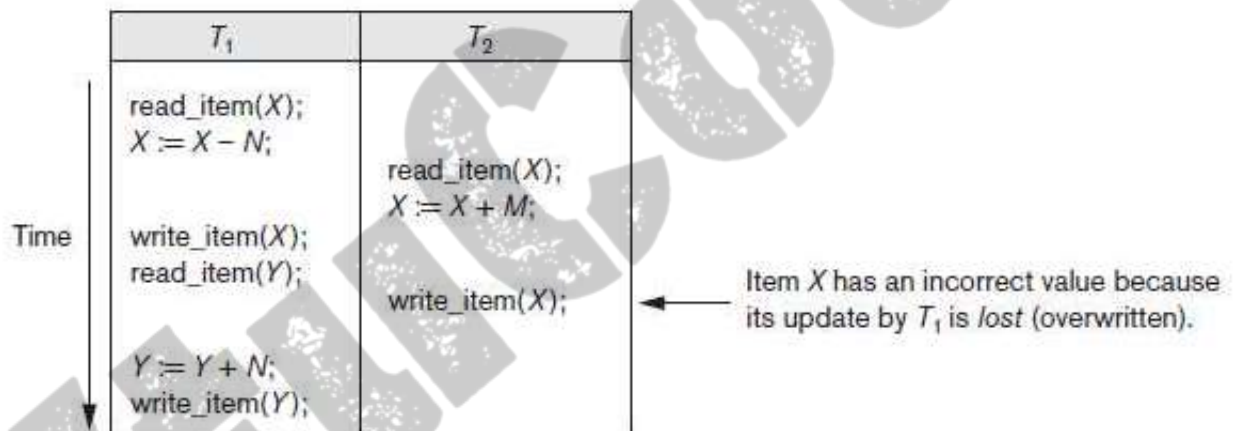
- Several problems can occur when concurrent transactions execute in an uncontrolled manner
- Example:
 - We consider an Airline reservation DB
 - Each records is stored for an airline flight which includes Number of reserved seats among other information.
 - Types of problems we may encounter:
 1. The Lost Update Problem
 2. The Temporary Update (or Dirty Read) Problem
 3. The Incorrect Summary Problem
 4. The Unrepeatable Read Problem



- Transaction T1
 - transfers N reservations from one flight whose number of reserved seats is stored in the database item named X to another flight whose number of reserved seats is stored in the database item named Y.
- Transaction T2
 - reserves M seats on the first flight (X)

1. The Lost Update Problem

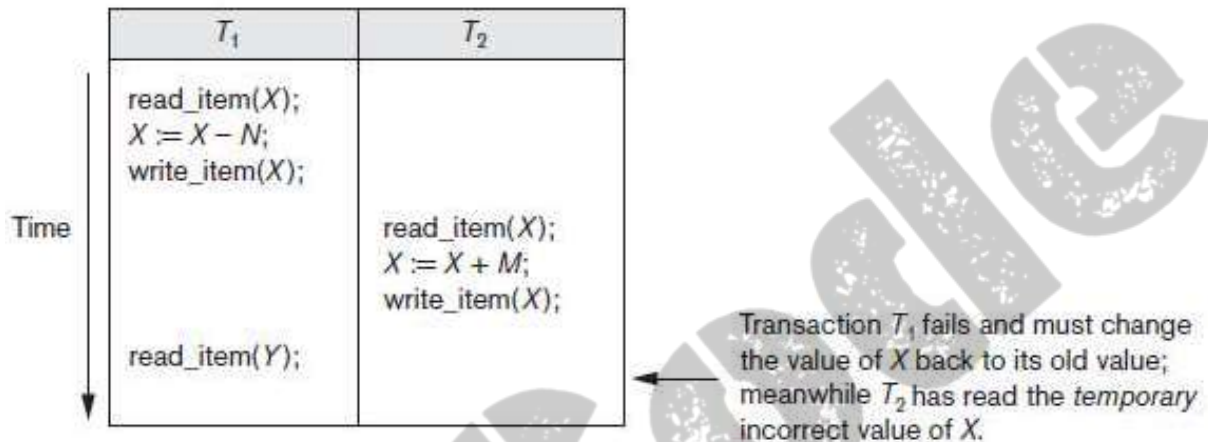
- occurs when two transactions that access the same DB items have their operations interleaved in a way that makes the value of some DB item incorrect
- Suppose that transactions T1 and T2 are submitted at approximately the same time, and suppose that their operations are interleaved as shown in Figure below



- Final value of item X is incorrect because T2 reads the value of X before T1 changes it in the database, and hence the updated value resulting from T1 is lost.
- For example:
 - X = 80 at the start (there were 80 reservations on the flight)
 - N = 5 (T1 transfers 5 seat reservations from the flight corresponding to X to the flight corresponding to Y)
 - M = 4 (T2 reserves 4 seats on X)
 - The final result should be X = 79.
- The interleaving of operations shown in Figure is X = 84 because the update in T1 that removed the five seats from X was lost.

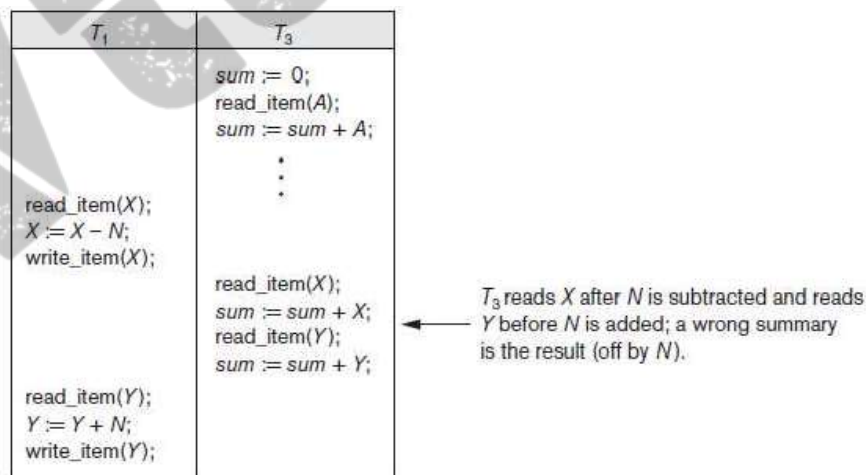
2. The Temporary Update (or Dirty Read) Problem

- occurs when one transaction updates a database item and then the transaction fails for some reason
- Meanwhile the updated item is accessed by another transaction before it is changed back to its original value



3. The Incorrect Summary Problem

- If one transaction is calculating an aggregate summary function on a number of db items while other transactions are updating some of these items, the aggregate function may calculate some values before they are updated and others after they are updated.



4. The Unrepeatable Read Problem

- Transaction T reads the same item twice and gets different values on each read, since the item was modified by another transaction T' between the two reads.
- for example, if during an airline reservation transaction, a customer inquires about seat availability on several flights
- When the customer decides on a particular flight, the transaction then reads the number of seats on that flight a second time before completing the reservation, and it may end up reading a different value for the item.

5.2.4 Why Recovery Is Needed

- Whenever a transaction is submitted to a DBMS for execution, the system is responsible for making sure that either
 1. All the operations in the transaction are completed successfully and their effect is recorded permanently in the database or
 2. The transaction does not have any effect on the database or any other transactions
- In the first case, the transaction is said to be committed, whereas in the second case, the transaction is aborted
- If a transaction fails after executing some of its operations but before executing all of them, the operations already executed must be undone and have no lasting effect.

Types of failures

1. A computer failure (system crash):

- A hardware, software, or network error occurs in the computer system during transaction execution
- Hardware crashes are usually media failures—for example, main memory failure.

2. A transaction or system error:

- Some operation in the transaction may cause it to fail, such as integer overflow or division by zero
- Also occur because of erroneous parameter values

3. Local errors or exception conditions detected by the transaction:

- During transaction execution, certain conditions may occur that necessitate cancellation of the transaction

- For example, data for the transaction may not be found

4. Concurrency control enforcement:

- The concurrency control may decide to abort a transaction because it violates serializability or several transactions are in a state of deadlock

5. Disk failure:

- Some disk blocks may lose their data because of a read or write malfunction or because of a disk read/write head crash.

6. Physical problems and catastrophes:

- refers to an endless list of problems that includes power or air-conditioning failure, fire, theft, overwriting disks or tapes by mistake
- Failures of types 1, 2, 3, and 4 are more common than those of types 5 or 6.
- Whenever a failure of type 1 through 4 occurs, the system must keep sufficient information to quickly recover from the failure.
- Disk failure or other catastrophic failures of type 5 or 6 do not happen frequently; if they do occur, recovery is a major task.

5.3 Transaction and System Concepts

5.3.1 Transaction States and Additional Operations

- A transaction is an atomic unit of work that should either be completed in its entirety or not done at all. For recovery purposes, the system keeps track of start of a transaction, termination, commit or aborts.
 - **BEGIN_TRANSACTION**: marks the beginning of transaction execution
 - **READ or WRITE**: specify read or write operations on the database items that are executed as part of a transaction
 - **END_TRANSACTION**: specifies that READ and WRITE transaction operations have ended and marks the end of transaction execution
 - **COMMIT_TRANSACTION**: signals a *successful end* of the transaction so that any changes (updates) executed by the transaction can be safely **committed** to the database and will not be undone
 - **ROLLBACK**: signals that the transaction has *ended unsuccessfully*, so that any changes or effects that the transaction may have applied to the database must be **undone**

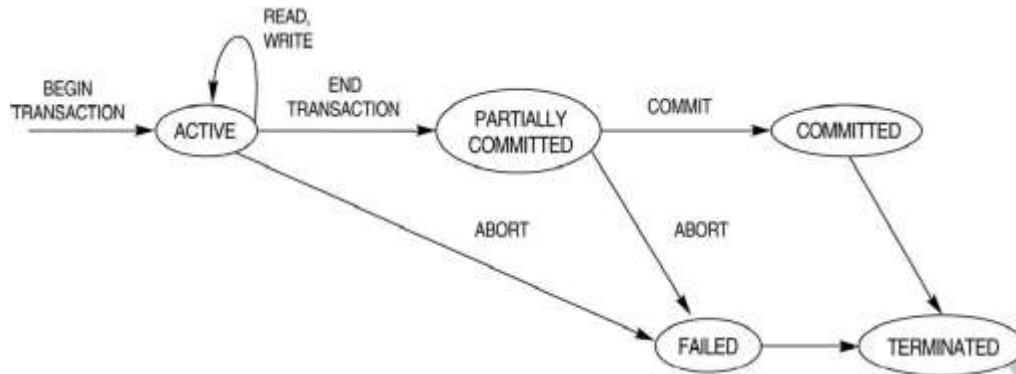


Figure: State transition diagram illustrating the states for transaction execution

- A transaction goes into **active state** immediately after it starts execution and can execute read and write operations.
- When the transaction ends it moves to **partially committed** state.
- At this end additional checks are done to see if the transaction can be committed or not. If these checks are successful the transaction is said to have reached commit point and enters **committed state**. All the changes are recorded permanently in the db.
- A transaction can go to the **failed state** if one of the checks fails or if the transaction is aborted during its active state. The transaction may then have to be rolled back to undo the effect of its write operation.
- Terminated state corresponds to the transaction leaving the system. All the information about the transaction is removed from system tables.

5.3.2 The System Log

- **Log or Journal** keeps track of all transaction operations that affect the values of database items
- This information may be needed to permit recovery from transaction failures.
- The log is kept on disk, so it is not affected by any type of failure except for disk or catastrophic failure
- one (or more) main memory buffers hold the last part of the log file, so that log entries are first added to the main memory buffer
- When the **log buffer** is filled, or when certain other conditions occur, the log buffer is *appended to the end of the log file on disk*.

- In addition, the log is periodically backed up to archival storage (tape) to guard against such catastrophic failures
- The following are the types of entries—called **log records**—that are written to the log file and the corresponding action for each log record.
- In these entries, T refers to a unique **transaction-id** that is generated automatically by the system for each transaction and that is used to identify each transaction:
 1. **[start_transaction, T]**. Indicates that transaction T has started execution.
 2. **[write_item, T, X, old_value, new_value]**. Indicates that transaction T has changed the value of database item X from old_value to new_value.
 3. **[read_item, T, X]**. Indicates that transaction T has read the value of database item X.
 4. **[commit, T]**. Indicates that transaction T has completed successfully, and affirms that its effect can be committed (recorded permanently) to the database.
 5. **[abort, T]**. Indicates that transaction T has been aborted.

5.3.3 Commit Point of a Transaction:

- **Definition a Commit Point:**
 - A transaction T reaches its **commit point** when all its operations that access the database have been executed successfully *and* the effect of all the transaction operations on the database has been recorded in the log.
 - Beyond the commit point, the transaction is said to be committed, and its effect is assumed to be permanently recorded in the database.
 - The transaction then writes an entry [commit,T] into the log.
- **Roll Back of transactions:**
 - Needed for transactions that have a [start_transaction,T] entry into the log but no commit entry [commit,T] into the log.

5.3.4 DBMS specific buffer Replacement policies

Domain Separation(DS) method

- DBMS cache is divided into separate domains, each handles one type of disk pages and replacements within each domain are handled via basic LRU page replacement.
- LRU is a **static** algorithm and does not adopt to dynamically changing loads because the number of available buffers for each domain is predetermined.
- **Group LRU** adds dynamically load balancing feature since it gives each domain a priority and selects pages from lower priority level domain first for replacement.

Hot Set Method:

- This is useful in queries that have to scan a set of pages repeatedly.
- The hot set method determines for each db processing algorithm the set of disk pages that will be accessed repeatedly and it does not replace them until their processing is completed.

The DBMIN method:

- uses a model known as QLSM (Query Locality set model), which predetermines the pattern of page references for each algorithm for a particular db operation
- Depending on the type of access method, the file characteristics, and the algorithm used the QLSM will estimate the number of main memory buffers needed for each file involved in the operation.

5.4 Desirable Properties of Transactions

- Transactions should possess several properties, often called the **ACID** properties
 - A **Atomicity**: a transaction is an atomic unit of processing and it is either performed entirely or not at all.
 - C **Consistency Preservation**: a transaction should be consistency preserving that is it must take the database from one consistent state to another.
 - I **Isolation/Independence**: A transaction should appear as though it is being executed in isolation from other transactions, even though many transactions are executed concurrently.
 - D **Durability (or Permanency)**: if a transaction changes the database and is committed, the changes must never be lost because of any failure.
- The **atomicity** property requires that we execute a transaction to completion. It is the responsibility of the transaction recovery subsystem of a DBMS to ensure atomicity.
- The preservation of **consistency** is generally considered to be the responsibility of the programmers who write the database programs or of the DBMS module that enforces integrity constraints.
- The **isolation** property is enforced by the concurrency control subsystem of the DBMS. If every transaction does not make its updates (write operations) visible to other transactions until it is committed, one form of isolation is enforced that solves the temporary update problem and eliminates cascading rollbacks
- **Durability** is the responsibility of recovery subsystem.

5.5 Characterizing Schedules Based on Recoverability

- **schedule (or history):** the order of execution of operations from all the various transactions
- **Schedules (Histories) of Transactions:** A schedule S of n transactions $T_1, T_2, \dots, T_n$ is a sequential ordering of the operations of the n transactions.
 - The transactions are interleaved
- Two operations in a schedule are said to **conflict** if they satisfy all three of the following conditions:
 - (1) they belong to *different transactions*;
 - (2) they access the *same item X* ; and
 - (3) *at least one* of the operations is a `write_item( $X$ )`
- **Conflicting operations:**
 - $r_1(X)$ conflicts with $w_2(X)$ } Read write conflict
 - $r_2(X)$ conflicts with $w_1(X)$ }
 - $w_1(X)$ conflicts with $w_2(X)$ Write conflict
 - $r_1(X)$ do not conflicts with $r_2(X)$

Schedules classified on recoverability:

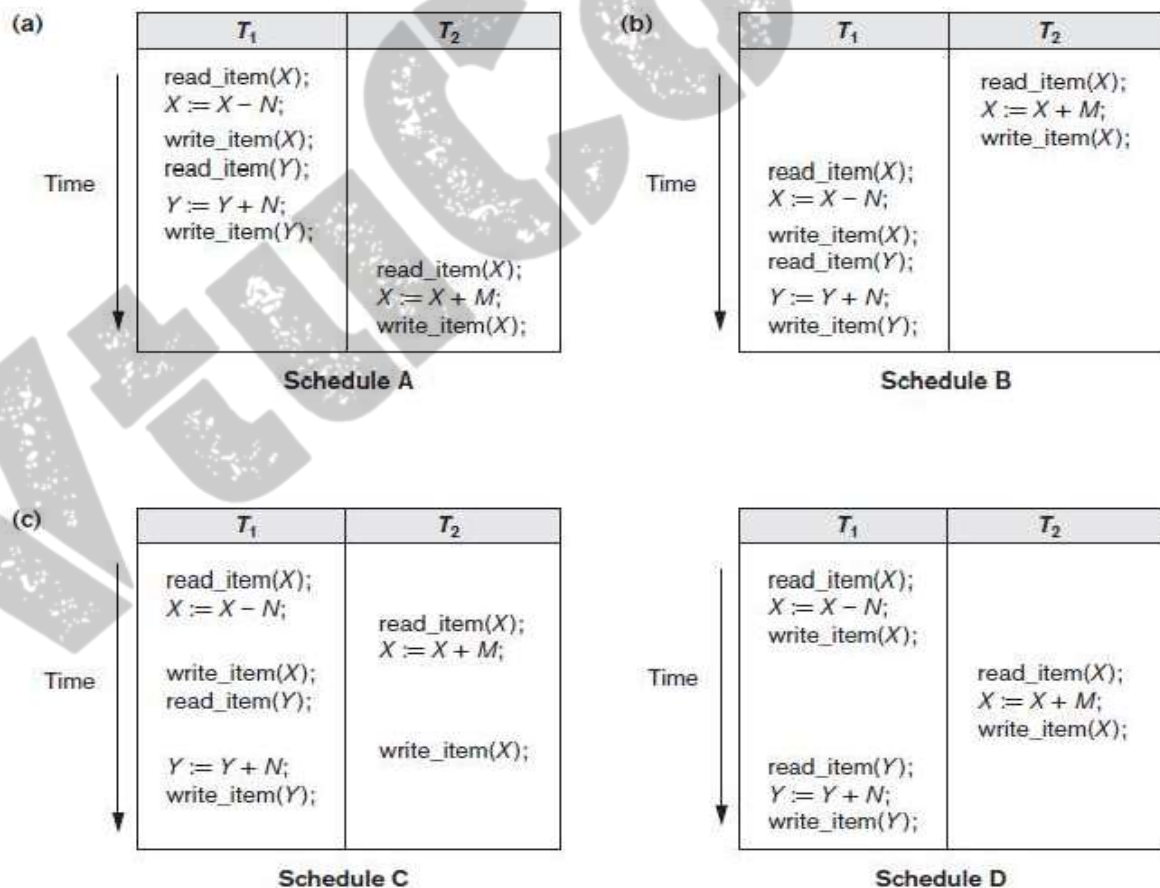
- **Recoverable schedule:**
 - One where no transaction needs to be rolled back.
 - A schedule S is recoverable if no transaction T in S commits until all transactions T' that have written an item that T reads have committed.
 - Example:
 - $S_c: r_1(X); w_1(X); r_2(X); r_1(Y); w_2(X); c_2; a_1;$
 - $S_d: r_1(X); w_1(X); r_2(X); r_1(Y); w_2(X); w_1(Y); c_1; c_2;$
- **Cascadeless schedule:**
 - One where every transaction reads only the items that are written by committed transactions.
- **Schedules requiring cascaded rollback:**
 - A schedule in which uncommitted transactions that read an item from a failed transaction must be rolled back.
- **Strict Schedules:**
 - A schedule in which a transaction can neither read or write an item X until the last transaction that wrote X has committed.

5.6 Characterizing Schedules Based on Serializability

- schedules that are always considered to be correct when concurrent transactions are executing are known as **serializable** schedules
- Suppose that two users—for example, two airline reservations agents—submit to the DBMS transactions T_1 and T_2 at approximately the same time. If no interleaving of operations is permitted, there are only two possible outcomes:
 - Execute all the operations of transaction T_1 (in sequence) followed by all the operations of transaction T_2 (in sequence).
 - Execute all the operations of transaction T_2 (in sequence) followed by all the operations of transaction T_1 (in sequence).

Figure 21.5

Examples of serial and nonserial schedules involving transactions T_1 and T_2 . (a) Serial schedule A: T_1 followed by T_2 . (b) Serial schedule B: T_2 followed by T_1 . (c) Two nonserial schedules C and D with interleaving of operations.



- **Serial schedule:**
 - A schedule S is serial if, for every transaction T participating in the schedule, all the operations of T are executed consecutively in the schedule.
 - Otherwise, the schedule is called nonserial schedule.
- **Serializable schedule:**
 - A schedule S is serializable if it is equivalent to some serial schedule of the same n transactions.
- **Result equivalent:**
 - Two schedules are called result equivalent if they produce the same final state of the database.
- **Conflict equivalent:**
 - Two schedules are said to be conflict equivalent if the order of any two conflicting operations is the same in both schedules.
- **Conflict serializable:**
 - A schedule S is said to be conflict serializable if it is conflict equivalent to some serial schedule S'.
- Being serializable is not the same as being serial
- Being serializable implies that the schedule is a correct schedule.
 - It will leave the database in a consistent state.
 - The interleaving is appropriate and will result in a state as if the transactions were serially executed, yet will achieve efficiency due to concurrent execution.

5.6.1 Testing conflict serializability of a Schedule S

For each transaction T_i participating in schedule S, create a node labeled T_i in the precedence graph.

For each case in S where T_j executes a `read_item(X)` after T_i executes a `write_item(X)`, create an edge ($T_i \rightarrow T_j$) in the precedence graph.

For each case in S where T_j executes a `write_item(X)` after T_i executes a `read_item(X)`, create an edge ($T_i \rightarrow T_j$) in the precedence graph.

For each case in S where T_j executes a `write_item(X)` after T_i executes a `write_item(X)`, create an edge ($T_i \rightarrow T_j$) in the precedence graph.

The schedule S is serializable if and only if the precedence graph has no cycles.

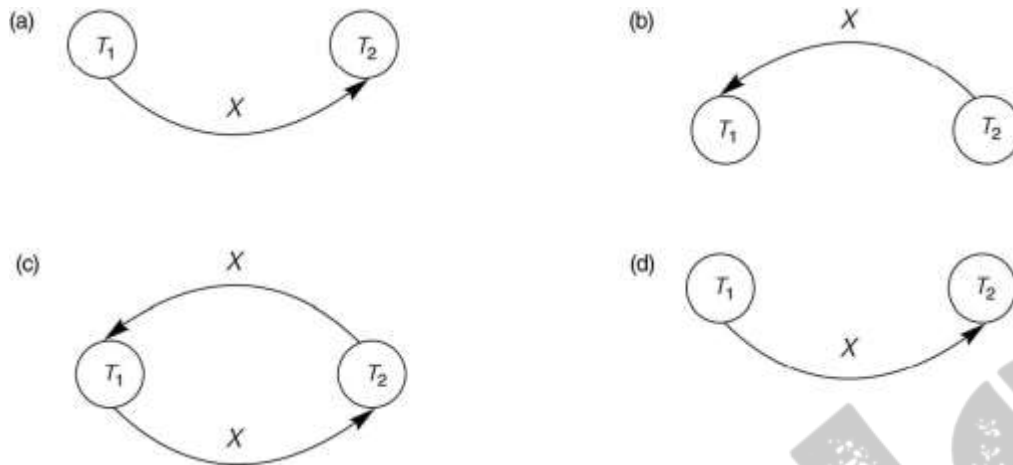


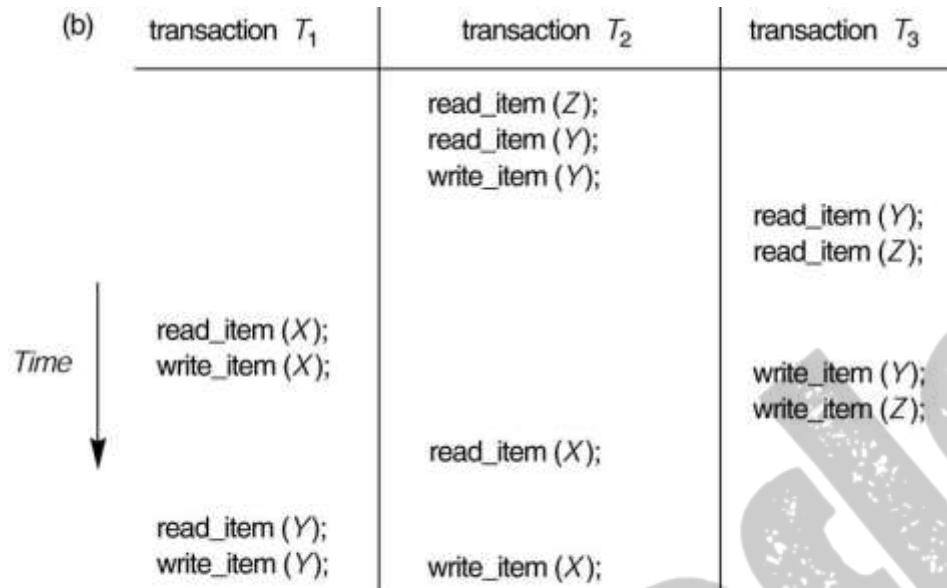
Fig: Constructing the precedence graphs for schedules A and D from fig 21.5 to test for conflict serializability.

- (a) Precedence graph for serial schedule A.
- (b) Precedence graph for serial schedule B.
- (c) Precedence graph for schedule C (not serializable).
- (d) Precedence graph for schedule D (serializable, equivalent to schedule A).

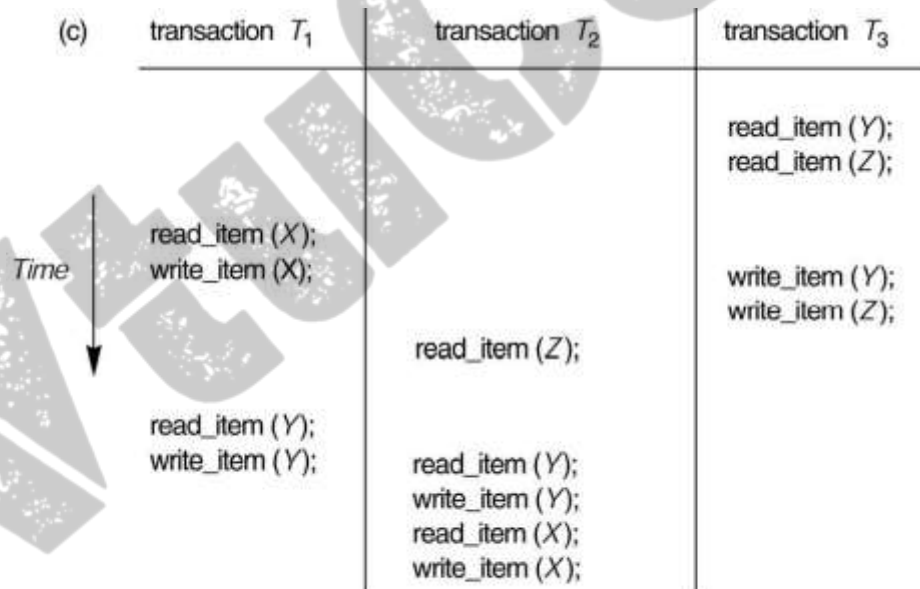
- Another example of serializability testing. (a) The READ and WRITE operations of three transactions T_1 , T_2 , and T_3 .

(a)

transaction T_1	transaction T_2	transaction T_3
<code>read_item (X);</code> <code>write_item (X);</code> <code>read_item (Y);</code> <code>write_item (Y);</code>	<code>read_item (Z);</code> <code>read_item (Y);</code> <code>write_item (Y);</code> <code>read_item (X);</code> <code>write_item (X);</code>	<code>read_item (Y);</code> <code>read_item (Z);</code> <code>write_item (Y);</code> <code>write_item (Z);</code>

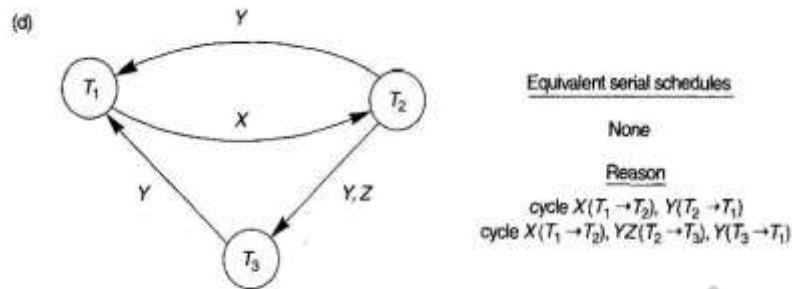


Schedule E

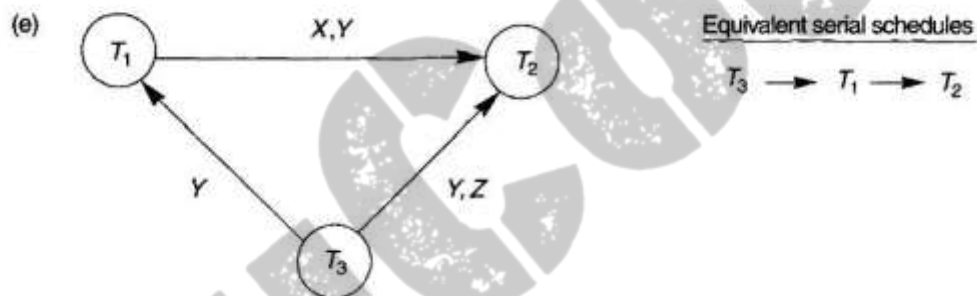


Schedule F

- Precedence graph for schedule E



- Precedence graph for schedule F



5.7 Transaction Support in SQL

- The basic definition of an SQL transaction is, it is a logical unit of work and is guaranteed to be atomic
- A single SQL statement is always considered to be atomic —either it completes execution without an error or it fails and leaves the database unchanged
- With SQL, there is no explicit Begin_Transaction statement. Transaction initiation is done implicitly when particular SQL statements are encountered
- Every transaction must have an explicit end statement, which is either a COMMIT or a ROLLBACK
- Every transaction has certain characteristics attributed to it and are specified by a SET TRANSACTION statement in SQL

- The characteristics are :
 - **The access mode**
 - can be specified as READ ONLY or READ WRITE
 - The default is READ WRITE
 - A mode of READ WRITE allows select, update, insert, delete, and create commands to be executed
 - A mode of READ ONLY, as the name implies, is simply for data retrieval.
 - **The diagnostic area size**
 - DIAGNOSTIC SIZE n , specifies an integer value n , which indicates the number of conditions that can be held simultaneously in the diagnostic area
 - These conditions supply feedback information (errors or exceptions) to the user or program on the n most recently executed SQL statement
- **The isolation level**
 - specified using the statement ISOLATION LEVEL <isolation>, where the value for <isolation> can be READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, or SERIALIZABLE
 - The default isolation level is SERIALIZABLE
 - The use of the term SERIALIZABLE here is based on not allowing violations that cause dirty read, unrepeatable read, and phantoms
 - If a transaction executes at a lower isolation level than SERIALIZABLE, then one or more of the following three violations may occur:
 1. **Dirty read.** A transaction T_1 may read the update of a transaction T_2 , which has not yet committed. If T_2 fails and is aborted, then T_1 would have read a value that does not exist and is incorrect.
 2. **Nonrepeatable read.** A transaction T_1 may read a given value from a table. If another transaction T_2 later updates that value and T_1 reads that value again, T_1 will see a different value.
 3. **Phantoms.** A transaction T_1 may read a set of rows from a table, perhaps based on some condition specified in the SQL WHERE-clause. Now suppose that a transaction T_2 inserts a new row that also satisfies the WHERE-clause condition used in T_1 , into the table used by T_1 . If T_1 is repeated, then T_1 will see a phantom, a row that previously did not exist.

Table 21.1 Possible Violations Based on Isolation Levels as Defined in SQL

Isolation Level	Type of Violation		
	Dirty Read	Nonrepeatable Read	Phantom
READ UNCOMMITTED	Yes	Yes	Yes
READ COMMITTED	No	Yes	Yes
REPEATABLE READ	No	No	Yes
SERIALIZABLE	No	No	No

```

EXEC SQL WHENEVER SQLERROR GOTO UNDO;
EXEC SQL SET TRANSACTION
    READ WRITE
    DIAGNOSTIC SIZE 5
    ISOLATION LEVEL SERIALIZABLE;
EXEC SQL INSERT INTO EMPLOYEE (Fname, Lname, Ssn, Dno, Salary)
    VALUES ('Robert', 'Smith', '991004321', 2, 35000);
EXEC SQL UPDATE EMPLOYEE
    SET Salary = Salary * 1.1 WHERE Dno = 2;
EXEC SQL COMMIT;
GOTO THE_END;
UNDO: EXEC SQL ROLLBACK;
THE_END: ... ;

```

- The transaction consists of first inserting a new row in the EMPLOYEE table and then updating the salary of all employees who work in department 2
- If an error occurs on any of the SQL statements, the entire transaction is rolled back This implies that any updated salary (by this transaction) would be restored to its previous value and that the newly inserted row would be removed.